

# UniMap: Unified Sizing and Remapping for ASIC Technology Mapping on GPU

Zhenxuan Xie

The Chinese University of Hong Kong  
zxjie24@cse.cuhk.edu.hk

Tianji Liu

Chinese Academy of Sciences  
liutianji@ict.ac.cn

Lixin Liu

The Chinese University of Hong Kong  
lxliu@cse.cuhk.edu.hk

Evangeline F.Y. Young

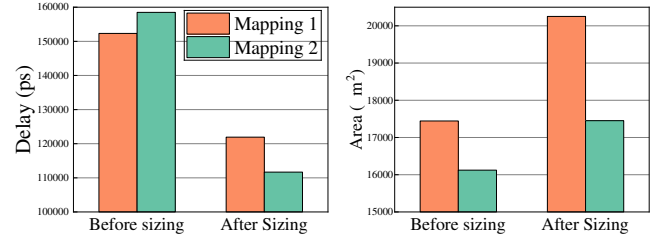
The Chinese University of Hong Kong  
fyyoung@cse.cuhk.edu.hk

## Abstract

ASIC technology mapping involves transforming a technology-independent Boolean network into an equivalent circuit utilizing cells from a specified library, a process that is vital in EDA. However, existing algorithms usually divide the process into two separate stages: mapping and sizing, resulting in a suboptimal overall quality. In this paper, we propose an incremental ASIC mapper that considers both stages jointly. This approach introduces an evaluation framework for the local region around each node, enabled by an on-the-fly incremental STA and AIG-based levelization. Moreover, two effective timing update strategies are incorporated into the incremental STA for deterministic parallelism. Finally, these innovations are integrated within a fine-grained parallelism framework on GPU. Experimental results on different benchmarks demonstrate that our method achieves on average 8% delay and 4% area reduction compared to ABC. Moreover, our method completes the netlist containing millions of gates within 10 seconds, achieving a 60.3× speedup over ABC.

## 1 Introduction

ASIC Technology mapping is a crucial step in logic synthesis, involving the transformation of a circuit from a technology-independent representation (e.g., And-Inverter Graph, AIG), into a technology-dependent network utilizing standard cells. This process is essential to optimize key performance metrics, including area, delay, and power [4, 5, 7, 11]. As the complexity of circuits continues to increase, the outcomes



**Figure 1: Comparison of different initial mappings for case “sqrt” in Nangate 45nm. Mapping 1 achieves better pre-sizing delay with higher area, while Mapping 2 achieves better post-sizing delay and area.**

of this phase can significantly impact subsequent physical design processes [24, 25, 31]. Therefore, obtaining efficient algorithms for high-quality technology mapping results has become increasingly important [6, 27].

Modern technology libraries provide multiple sizes for each standard cell. In principle, mapping and sizing can be performed simultaneously; however, this leads to two challenges. First, the joint search space becomes prohibitively large due to the combination of structural and sizing choices. Second, accurate sizing depends on reliable timing information (e.g., slew and load), which is difficult to obtain without a stable mapping solution. Therefore, traditional flows decompose the problem into two stages: mapping with minimum-sized cells to obtain a valid structure, followed by gate sizing to refine timing. While effective, this separation restricts the solution space and may lead to suboptimal results.

Recently, several methods have been proposed [4, 26] to solve the first step. SLAP [28] employed supervised learning for ranking cuts. LEAP [10] proposed a machine learning-assisted approach to further reduce used cuts. The work proposed by [18] try to predict the gate delay before mapping. Liu et al. [17] proposed a heuristic to estimate input transitions and output loads according to the structure around the nodes. These approaches can improve the quality of the

The work described in this paper was substantially supported by a grant from the Research Grants Council of the Hong Kong Special Administrative Region, China (Project Reference Number: T46-415/25-R).



This work is licensed under a Creative Commons Attribution 4.0 International License.

ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2873-0/2026/11

<https://doi.org/10.1145/3831252.3833937>

initial mapping, but this cannot guarantee an overall high-quality result after gate sizing. Figure 1 presents experimental results using different initial mappings followed by the same sizing algorithm. Although Mapping 1 achieves better delay than Mapping 2 before sizing with an area penalty, its delay and area both become worse after sizing. In one word, solving the mapping problem with separate two steps will shrink the solution space, leading to suboptimal results.

In addition, GPU computing has emerged as an effective solution for addressing runtime scalability challenges in EDA, owing to its high degree of parallelism and substantial memory bandwidth. Numerous studies have demonstrated that GPU-accelerated algorithms can deliver significant efficiency gains across a wide range of tasks [16, 19–21, 23, 30, 32]. Many of these algorithms are related to logic synthesis due to the level-wise structure of Boolean circuits. Notably, FineMap [21] proposed a data-parallelism algorithm that leverages GPU to accelerate LUT technology mapping, achieving a  $59.8\times$  speedup over the public synthesis tool ABC [4].

In this paper, we propose an incremental mapping algorithm named UniMap, which simultaneously considers sizing and remapping for an initial mapping solution. This enables a more comprehensive exploration of the solution space, leading to improved overall mapping quality. Furthermore, the algorithm is designed to be highly parallelizable, which can leverage the computational capabilities of modern GPU resources to enhance efficiency and scalability. Our contributions can be summarized as follows:

- We introduce an evaluation framework capable of uniformly assessing the impact of sizing and remapping.
- We define local regions around each node and design two gain functions for global delay optimization and area reduction, enabling efficient trade-off control between performance metrics.
- We propose an efficient incremental static timing analysis (STA) method that guarantees timing accuracy and achieves deterministic parallel execution through load-timing and timing cache strategies.
- We develop a novel levelization algorithm based on the mapping AIG that enforces correct timing propagation order while preserving remapping flexibility for opposite-phase inverter transformations.
- The overall algorithm is implemented with fine-grained parallelism on GPU to maximize hardware utilization.

We evaluate our algorithm on the EPFL and IWLS 2005 benchmarks with the Nangate 45nm library [1]. Compared with ABC [4], our method reduces delay and area by 8% and 4% on average, with up to 13% and 20% improvements, respectively, while achieving a  $60.3\times$  GPU speedup. Ablation studies further demonstrate the effectiveness of jointly optimizing gate sizing and remapping.

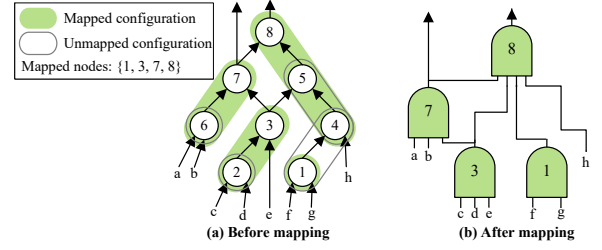


Figure 2: Illustration of ASIC mapping.

## 2 PRELIMINARIES

### 2.1 Background

A *Boolean network* can be represented as a directed acyclic graph (DAG) where each node corresponds to a Boolean function, and the edges signify the input/output signals of these nodes. The *transitive fanins* and *transitive fanouts* of a node represent its predecessors and successors, respectively. Specifically, direct predecessors and successors are referred to as *fanins* and *fanouts*. *Primary inputs* (PIs) and *primary outputs* (POs) serve as sources and sinks of the DAG.

An *And-Inverter Graph* (AIG) is a Boolean network where all nodes are two-input AND gates with optionally complemented fanin signals. *Phase* indicates whether the signal of a node is used directly (positive) or inverted (negative), and  $\neg n$  denotes the opposite phase of node  $n$ .

A *technology library* contains pre-designed cells with detailed characteristics, including area, pin capacitance, slew, and delay lookup tables. *Slew* describes the output voltage transition rate. Before physical design, a node's input slew is the maximum output slew of its fanins and is typically set to 0 for PIs. *Load* is the summation of the pin capacitance of all fanouts connected to a given node. *Delay lookup-table* is a table that describes the delay from an input to an output of a cell, contingent upon the respective input slew and load. Their relationships can be expressed as

$$\text{delay} = f(\text{input slew}, \text{load}) \quad (1)$$

Similarly, *slew lookup-table* is for calculating output slew.

A *cell network* is a Boolean network in which each node corresponds to a specific function cell provided by the technology library. A key aspect of the mapping process is the definition of the arrival time for each node. The arrival time of a node is defined as the longest path from any PI to that node, computed recursively as

$$A(n) = \max_{i \in \text{fanins}(n)} (d(i) + A(i)) \quad (2)$$

where  $d(i)$  denotes the gate delay from fanin  $i$  to the output  $n$ . The *global delay* of network is the maximum arrival time of all POs, while the area is the sum of all the gates.

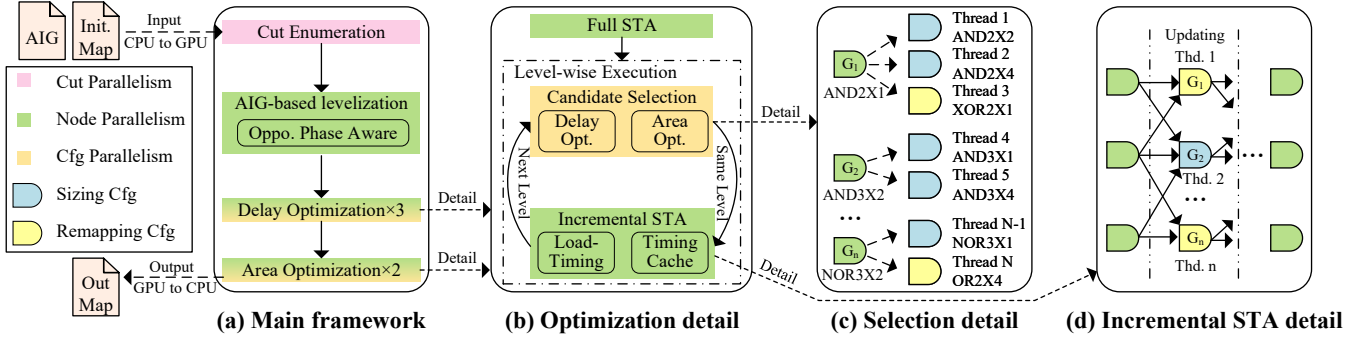


Figure 3: The framework of UniMap.

#### Algorithm 1 Overall Flow of UniMap

**Input:** AIG  $G$ , initial mapping  $M$

- 1:  $cut \leftarrow \text{CUTENUMERATION}(G, N)$  ▷ Sec. 3.2
- 2:  $(Levels, n) \leftarrow \text{AIGBASEDLEVELIZATION}(G, N)$  ▷ Sec. 3.5
- 3: **for**  $iter = 0$  **to**  $5$  **do** ▷ Delay opt.  $\times 3$ , area opt.  $\times 2$
- 4:    $\text{FULLSTA}()$
- 5:   **for**  $i = 0$  **to**  $n$  **do** ▷ Iterate level by level
- 6:     **for all** node  $g \in Levels[i]$  **in parallel do**
- 7:        $C_{size}[g] \leftarrow \text{SIZINGCANDIDATE}(g)$
- 8:        $C_{remap}[g] \leftarrow \text{REMAPPINGCANDIDATE}(g, cut)$
- 9:        $C_{merged} \leftarrow \text{MERGEALLCANDIDATES}(C_{size}, C_{remap})$
- 10:      **for all** candidate  $c \in C_{merged}$  **in parallel do**
- 11:        $Gain[c] \leftarrow \text{EVALUATION}(c)$  ▷ Sec. 3.3
- 12:      **for all** node  $g \in Levels[i]$  **in parallel do**
- 13:        $c_{best} \leftarrow \arg \max_{c \in (C_{size}[g] \cup C_{remap}[g])} Gain[c]$
- 14:        $\text{UPdatenode}(g, c_{best})$
- 15:       $\text{INCREMENTALSTA}(g)$  ▷ Sec. 3.4

For a node  $n$  in a Boolean network, a *cut* of  $n$  is a set of nodes such that any path from a PI to  $n$  passes through at least one node in the set. The *logic cone* associated with a cut of  $n$  includes the node  $n$  itself, along with the intersection of all transitive fanins of  $n$  and all transitive fanouts of the nodes in the cut. A *configuration* of a cut corresponds to a cell from the technology library that is functionally equivalent to the logic cone of the cut.

ASIC mapping transforms a Boolean network of a particular representation into a cell network as shown in Figure 2. A final configuration for each node is computed, and a subset of these nodes is selected for mapping.

## 2.2 Sizing and Remapping

In a mapped cell network, each node corresponds to a standard cell from the technology library.

**Gate sizing** refers to replacing a cell with another size of the same type in order to optimize timing or area.

**Remapping** modifies the implementation of a node by selecting an alternative cell (i.e., a different logic structure) derived from its AIG representation. In this work, remapping is restricted to be *one-to-one*, meaning that each mapped node is replaced by exactly one alternative configuration, without introducing new nodes.

To ensure mapping legality and reduce complexity, we only consider configurations whose fanins are already mapped in the initial solution. Consequently, remapping does not activate previously unmapped AIG nodes, and the total number of mapped nodes remains unchanged.

**Incremental mapping** in this work refers to jointly applying gate sizing and remapping on an existing mapping solution to improve overall delay and area, while preserving mapping legality and structural consistency.

## 3 PROPOSED ALGORITHMS

This section presents our proposed incremental mapping algorithm, UniMap. We first provide an overview of the algorithmic flow. Next, we describe how the local region of a candidate configuration is defined and how its impact is evaluated using two distinct gain functions. We then introduce an efficient incremental STA method that incorporates two strategies to ensure deterministic behavior under parallel execution. Following this, we present an AIG-based levelization that preserves correct timing propagation order and supports inverter remapping. Finally, we discuss the details of our GPU-based implementation.

### 3.1 Overview

The framework of UniMap is illustrated in Figure 3 and the overall flow is shown in Alg. 1. The algorithm consists of four main stages: cut enumeration, AIG-based levelization, delay optimization, and area optimization. Before each optimization iteration, a full STA is performed to obtain accurate timing information for the current mapping solution. During

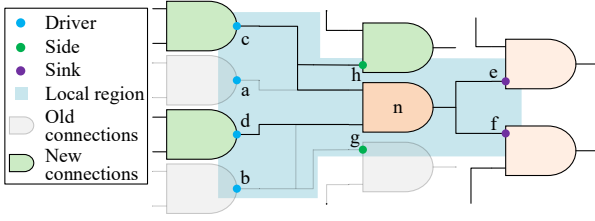


Figure 4: Illustration of local region

each iteration, an incremental STA is employed to update timing based on local changes.

**Cut Enumeration:** This is a standard phase in technology mapping. For each node, at most  $C$  priority  $k$ -feasible cuts are computed, where  $k$  denotes the maximum number of leaves in each cut. More details can be found in [8, 28]. The priority cuts for each node will be used in the remapping.

**AIG-based Levelization:** Prior to optimization, the netlist is leveled to ensure correct level-by-level evaluation order. Furthermore, this step also guarantees that the timing propagation of incremental STA follows the correct order.

**Delay Optimization:** In this phase, the timing gain of each candidate configuration within a local region is evaluated to optimize the overall delay. Both timing and area gains are jointly considered to balance delay and area efficiency.

**Area Optimization:** This phase focuses on reducing area without increasing global delay. The local worst-negative slack (WNS) is computed to assess each candidate’s impact on global timing, ensuring that area reduction does not compromise timing closure.

### 3.2 Cut Enumeration

Cut enumeration prepares the candidate configurations for each node. Previous ASIC mappers [8, 28] typically adopt four criteria to select priority cuts: mappability, area flow, number of leaves, and delay. However, additional considerations are required in incremental mapping. Specifically, the leaves of a configuration must be mapped to ensure legality, but traditional cut enumeration may generate cuts that violate this constraint.

To address this, we adopt the average number of mapped leaves as the second criterion following mappability. It is defined as:

$$\text{AvgMap}(c) = \frac{\sum_{i \in \text{leaves}(c)} (\text{IsMap}(i) + \text{IsMap}(\neg i))}{|\text{leaves}(c)|} \quad (3)$$

where  $\text{IsMap}(i)$  indicates whether node  $i$  is mapped in the initial mapping. Since each node has two phases, the mapped count is defined as the sum over both phases. The remaining criteria are applied as tie-breakers when the top two metrics are the same.

### 3.3 Candidate Selection

Evaluating candidate configurations is a crucial part of the algorithm, as it directly determines the mapping quality. For gate sizing, prior studies typically employ lagrangian relaxation-based cost functions or differentiable-based methods for optimization [12, 13, 29]. However, these approaches cannot be utilized in incremental mapping, since remapping will invalidate the Karush-Kuhn-Tucker conditions [9, 13] and disrupt the subgradient continuity. Therefore, we design a new evaluation framework for incremental mapping.

Changing the configuration of one node will affect the timing of the entire circuit, as both the input/output slew and arrival time are altered. However, performing a full STA to evaluate each candidate is computationally prohibitive. To address this, we restrict the evaluation scope to a smaller *local region*. As illustrated in Figure 4, we define the local region  $r$  as the subcircuit surrounding the node  $n$ . Pins  $a$ ,  $b$ ,  $c$ , and  $d$  are drivers, where  $a$ ,  $b$  correspond to the old drivers, while  $c$ ,  $d$  are the new ones. Pins  $e$ ,  $f$  serve as the sinks, and pins  $g$ ,  $h$  represent the old and new sides, respectively. These pins share nets connected to node  $n$ , and their timings are the most significantly influenced by the configuration update. Note that for gate sizing, new drivers and sides are the same as the old ones.

Assuming the timing of all transitive fanins is accurate, the new arrival time and output slew of the drivers can be computed by the lookup tables under the updated load. Although a driver  $d_a$  may serve as a transitive fanin of another driver  $d_b$ , which could introduce minor inaccuracies in the input slew of  $d_b$ , the effect is negligible since such cases are rare and have limited impact.

Once the new arrival time and output slew of the drivers are obtained, the timing can be propagated to the newly mapped gate of the configuration. Since wire delay is ignored in our model, the slew and arrival time of the side and sink pins are identical to those of the corresponding drivers and the mapped gate, respectively.

Furthermore, the local WNS of a region  $r$  is computed as the minimum slack among all the outputs in the local region:

$$\text{LWNS}(r) = \min_{o \in \text{outputs}(r)} (\text{Required}(o) - \text{Arrival}(o)) \quad (4)$$

Here, the outputs refer to the side and sink pins, as they propagate timing information to the downstream of the local region. And the required time is precomputed during the full STA before optimization, with the required time of POs set to the global delay.

To optimize the overall performance of the mapped circuit, we design two gain functions to evaluate each candidate configuration, one for delay optimization and the other for area optimization.

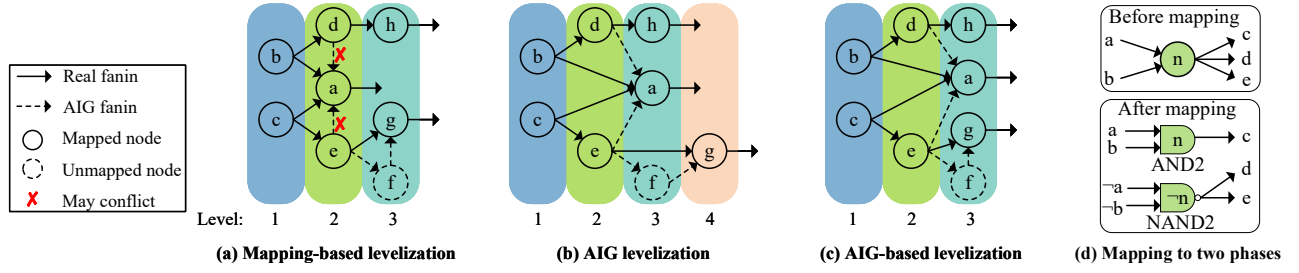


Figure 5: Illustration of AIG-based levelization and mapping of both phases. Mapping-based levelization may lead timing conflict, AIG levelization may introduce more levels, AIG levelization solves both issues.

**3.3.1 Delay optimization.** During this phase, our objective is to minimize the overall circuit delay while keeping area increase within a reasonable range. Given the defined local region, an intuitive way is to minimize the arrival time of the outputs. In addition, the output slew should also be considered, as it affects the delay of downstream gates, although its impact is typically secondary. However, since the global delay is determined by the maximum arrival time of the circuit, outputs with a smaller slack will have greater influence on the overall performance. Therefore, each output is weighted according to its slack to reflect its timing criticality. Based on these thoughts, we define the delay gain function as:

$$DG_{delay}(r) = \sum_{o \in \text{outputs}(r)} W_o \cdot (\Delta \text{Arrival}(o) + \alpha \cdot \Delta \text{Slew}(o)) \quad (5)$$

where  $\Delta \text{Arrival}(o)$  and  $\Delta \text{Slew}(o)$  represent the difference in arrival time and slew after updating the configuration, respectively. The parameter  $\alpha$  is a weighting factor that balances the influence of arrival time and slew, and is set to 0.1 in our experiments, as the effect of output slew is indirect and relatively minor compared to arrival time. The weight  $W_o$  is defined as:

$$W_o = W_{base} + \text{LWNS}(r) - \min(\text{Slack}_{old}(o), \text{Slack}_{new}(o)) \quad (6)$$

where  $\text{Slack}_{old}(o)$  and  $\text{Slack}_{new}(o)$  denote the slack of output  $o$  before and after the configuration change, respectively.  $W_{base}$  represents a base weight to prevent negative weight. This formulation measures the timing criticality of each output relative to local WNS. In essence, the outputs whose slacks are close to the local WNS are considered more critical and thus assigned higher weights. Moreover, by taking the minimum of the old and new slacks, we ensure that if an output is timing-critical before or after the configuration update, its timing will be properly emphasized. By considering area overhead, the overall gain function is defined as:

$$G_{delay}(r) = DG_{delay}(r) + \beta \cdot \Delta \text{Area}(r) \quad (7)$$

where  $\Delta \text{Area}(r)$  is the difference in area after the configuration update,  $\beta$  is a weight that balances trade-offs.

**3.3.2 Area optimization.** For area optimization phase, our objective is to minimize the overall area without degrading the global timing performance. Since the local WNS can be computed within each local region, a non-negative local WNS generally indicates that the configuration change will not increase the global delay. Therefore, the delay gain function is defined as:

$$DG_{area}(r) = \min(0, \text{LWNS}_{new}(r)) - \min(0, \text{LWNS}_{old}(r)) \quad (8)$$

where  $\text{LWNS}_{old}(r)$  and  $\text{LWNS}_{new}(r)$  represent the local WNS of region  $r$  before and after configuration update, respectively. Similarly to delay optimization, the overall gain function of this phase will also combine with the weighted area gain as shown in Equation 7. This gain function encourages selecting configurations that reduce area when the timing is non-critical (i.e.,  $\text{LWNS} \geq 0$ ), while also attempting to fix the timings that may increase global delay.

### 3.4 Deterministic Incremental STA

Incremental STA is employed to update timing so that the nodes in downstream levels can be evaluated correctly. Since our algorithm proceeds in a topological level-wise manner, it is sufficient to propagate the new timing only to the outputs of local region, following each configuration update. However, directly implementing incremental STA in parallel may lead to non-deterministic results. This issue arises from two cases: (1) two local regions may share the same driver, and (2) one driver may be the fanin of another driver.

The first case causes the new load of shared driver to become non-deterministic during timing computation. To address this, we design a load-timing strategy to resolve the operation order problem. Specifically, the load of driver is first computed by atomic operations. Once all loads are updated, the new values are then used to calculate the new timing of the drivers. This strategy ensures the load of each driver remains deterministic during timing computation.

The second case can lead to non-deterministic input timing for drivers during timing computation. For example, node

$a$  is the driver of configuration  $c_1$ , and  $b$  is the driver of configuration  $c_2$ , with  $a$  being a fanin of  $b$ . If both  $c_1$  and  $c_2$  are updated simultaneously, the timing of  $a$  and  $b$  will be recomputed and propagated to their fanouts. However, since the timing of  $b$  partially depends on  $a$ , and their update order is non-deterministic, leading to inconsistent timing results of  $b$ . To mitigate this, we consistently use the old input timing of drivers. In practice, we create a timing cache to store the timing before performing incremental STA. Then, the new timing of drivers is always computed based on the cached timing. Although this approach slightly affects accuracy, the impact is usually negligible while reducing runtime and ensuring determinism. For efficiency, we only need to cache the input timing of related drivers rather than all nodes.

### 3.5 AIG-based Levelization

We need levelize the netlist to determine the evaluation order and enable parallel evaluation before optimization. For gate sizing only, the levelization can be performed by mapping-based levelization method [14] based on the initial circuit.

However, remapping may change node fanin/fanout relationships and invalidate the initial mapped topological order. As shown in Figure 5(a), node  $a$  has fanins  $b, c$  in the initial mapping but may be remapped to use  $d, e$  from the AIG. Standard levelization may evaluate  $a$  before the timing information of  $d, e$  is updated.

To address this issue, we propose an AIG-based levelization method. If node  $a$  is a transitive fanin of node  $b$  in the AIG, it can potentially become a fanin of  $b$  after remapping. Therefore, levelizing based on the AIG guarantees that all possible fanins are evaluated before their dependent nodes. To avoid unnecessary level increases caused by unmapped AIG nodes (e.g., node  $f$  in Figure 5(b)), we skip them during levelization as shown in Figure 5(c).

Algorithm 2 shows the AIG-based levelization. We first set PIs as the first-level nodes (lines 1). Then, we define a function for propagating each node to its fanouts (lines 3-10). Specifically, for each fanout, its remaining fanin count is decreased by one (line 5), and we check whether all its fanins are recorded (line 6). If the node is mapped, it will be added to the next-level node set (lines 7-8). Otherwise, it will be inserted into unmapped node set (lines 9-10).

The levelization is performed iteratively as long as the current node set  $CN$  is not empty (lines 11-20). For each node in  $CN$  (line 12), its level is assigned as the current level index  $CL$  (line 13), and it is propagated to its fanouts (line 14). For the unmapped nodes, we propagate them to their fanouts until no unmapped nodes remaining to be handled (lines 16-19). Finally,  $CN$  is replaced by the next-level node set  $CN'$ , and  $CL$  is increased by one (line 20).

---

#### Algorithm 2 AIG-based levelization

---

**Input:** Nodes  $N$ , mapping  $M$ , fanin number  $IN$ , fanout  $FO$

**Output:** Node level  $L$

```

1:  $CN \leftarrow \{n \in N : IN[n] = 0\}$  ▷ Initialize PIs
2:  $CL \leftarrow 0, CN' \leftarrow \{\}, UM \leftarrow \{\}, UM' \leftarrow \{\}$ 
3: function PROPAGATENode( $n$ ) ▷ Propagate to fanouts
4:   for  $o \in FO[n]$  do
5:      $IN[o] \leftarrow IN[o] - 1$ 
6:     if  $IN[o] = 0$  then
7:       if  $M[o] \neq -1$  then ▷ Mapped nodes
8:          $CN' \leftarrow CN' \cup \{o\}$ 
9:       else ▷ Unmapped nodes
10:         $UM' \leftarrow UM' \cup \{o\}$ 
11: while  $CN \neq \emptyset$  do
12:   for  $n \in CN$  in parallel do
13:      $L[n] = CL$  ▷ Assign node level
14:     PROPAGATENode( $n$ )
15:    $UM \leftarrow UM', UM' \leftarrow \{\}$ 
16:   while  $UM \neq \emptyset$  do ▷ Deal with unmapped nodes
17:     for  $n \in UM$  in parallel do
18:       PROPAGATENode( $n$ )
19:      $UM \leftarrow UM', UM' \leftarrow \{\}$ 
20:    $CN \leftarrow CN', CN' \leftarrow \{\}, CL \leftarrow CL + 1$ 

```

---

As shown in Figure 5 (d), using AIG to remapping offers an additional advantage: two phases of a node can be easily identified in the circuit, allowing one phase to be remapped as an inverter connected with the other phase. However, it is necessary to determine their level order before remapping, otherwise, if both phases are assigned to the same level, their timing cannot be propagated correctly under parallel execution, which is similar to the conflict case in Figure 5 (a). For the case where one phase has been mapped as an inverter in the initial mapping, we simply preserve the same topological order as initial mapping. In contrast, if both phases are mapped as non-inverter gates, we assign one phase higher level to ensure it can potentially be remapped as inverter.

Since remapping will alter the fanins of a node, positioning the phase with smaller arrival time at a lower level is better. Otherwise, its arrival time will increase if remapped as an inverter. However, the arrival time will change during optimization, we instead adopt the mapping level as a surrogate measure since the mapping level of two phases may be different. Specifically, we first perform mapping-based levelization [14] on initial mapping. If the level of phase  $p$  is smaller than that of phase  $\neg p$ , we assign levels  $i, i + 1$  to phases  $p, \neg p$  during the AIG-based levelization, respectively. otherwise, the assignment is reversed.

**Table 1: Comparison of results on Nangate 45nm**

| Benchmarks     | AIG Statistics |        | Initial Solutions |                   |          | ABC <i>upsize; dsize</i> |                   |              | UniMap (Ours) |                   |             |
|----------------|----------------|--------|-------------------|-------------------|----------|--------------------------|-------------------|--------------|---------------|-------------------|-------------|
|                | #Nodes         | Levels | Delay(ps)         | Area( $\mu m^2$ ) | #Gates   | Delay(ps)                | Area( $\mu m^2$ ) | Time(s)      | Delay(ps)     | Area( $\mu m^2$ ) | Time(s)     |
| log2_8xd       | 8207360        | 444    | 10165.16          | 4759774           | 3923712  | 8473.78                  | 4954204           | 673.78       | 7605.3        | 5011797           | 8.92        |
| mem_ctrl_8xd   | 11990016       | 114    | 5919.44           | 7341293           | 6899712  | 2783.94                  | 7364803           | 557.81       | 2436.42       | 7461619           | 16.86       |
| hyp_6xd        | 13717440       | 24801  | 919349.55         | 10765705          | 9588928  | 581998.00                | 11973173          | 7186.63      | 506226.38     | 11178424          | 161.71      |
| div_8xd        | 14655232       | 4372   | 84361.52          | 12458299          | 12996608 | 73158.06                 | 13878486          | 1310.33      | 63557.92      | 14279186          | 34.08       |
| sin_10xd       | 5545984        | 225    | 5410.95           | 3819368           | 3331072  | 4726.76                  | 3965063           | 1661.81      | 4166.64       | 3925870           | 6.66        |
| sqrt_7xd       | 3151104        | 5058   | 344464.14         | 2864900           | 2411520  | 156101.92                | 3339351           | 3826.85      | 144434.14     | 2847451           | 36.91       |
| voter_9xd      | 7044096        | 70     | 1924.63           | 7368940           | 6564352  | 1728.93                  | 7513064           | 524.62       | 1599.3        | 6027857           | 8.75        |
| square_9xd     | 9463808        | 250    | 4255.40           | 6489821           | 5784064  | 3762.82                  | 6723194           | 482.63       | 3498.53       | 6618522           | 8.16        |
| multiplier_9xd | 13855744       | 274    | 5577.44           | 8465830           | 7198208  | 4791.41                  | 9042197           | 1500.34      | 4746.36       | 8394874           | 10.56       |
| ac97_ctrl_10xd | 14610432       | 12     | 1355.24           | 8860923           | 6478848  | 426.09                   | 8889797           | 626.53       | 425.54        | 8836954           | 9.65        |
| vga_lcd_5xd    | 4054752        | 24     | 147372.76         | 2217707           | 1500256  | 37174.38                 | 2217853           | 49.62        | 37193.51      | 2216729           | 4.68        |
| Geomean        | 8095313        | 346    | 18348.45          | 6064381           | 5179662  | 11251.68                 | 6388101           | 886.52       | 10380.55      | 6103356           | 14.70       |
| Ratio          |                |        | <b>1.63</b>       | <b>0.95</b>       |          | <b>1.00</b>              | <b>1.00</b>       | <b>60.30</b> | <b>0.92</b>   | <b>0.96</b>       | <b>1.00</b> |

“\_nxd” means that the benchmark is generated by enlarging the original one using ABC *double n* times.

### 3.6 GPU Implementation

A suitable job division and allocation on GPU can significantly improve computational efficiency. For cut enumeration and full STA, we follow the methods proposed in [14, 21]. For incremental STA and node configuration update, we adopt a node-level parallelism strategy as shown in Figure 3 (d), where each thread is responsible for processing one node.

Candidate configuration evaluation is the most time consuming part. We employ fine-grained parallelism, assigning one thread to each candidate configuration. However, if each block only processes the configurations from one node [21], workload imbalance may occur due to the varying number of configurations per node. To maximize parallelism, we assign each block a batch of 128 candidate configurations, which may come from different nodes. To implement this, each thread is assigned with a unique candidate configuration first, and the mapping relationship between configurations and nodes is also recorded. Finally, all threads are launched simultaneously as shown in Figure 3 (c).

## 4 Experimental Results

To evaluate the performance of the proposed GPU-based incremental mapper, we conduct comprehensive experiments on 11 public benchmark circuits. We compare UniMap with the industrial-strength tool ABC [4]. Other open-source gate sizers are not included in the comparison for two reasons: (1) they typically rely on post-placement physical information (e.g., DEF/SPEF) which is unavailable in our purely combinational synthesis stage; (2) traditional sizing benchmarks lack the pre-mapped AIGs essential for exploring our remapping space. Therefore, ABC serves as the most robust baseline for comparison. In addition, we perform ablation studies and

procedure analysis to further examine the effectiveness of our proposed approaches.

### 4.1 Experimental Protocol

Our proposed GPU incremental ASIC mapping engine, UniMap, is implemented in about 4,000 lines CUDA C/C++. All experiments are conducted on an Ubuntu 18.04 server equipped with an Intel Xeon Silver 4114 CPU and an NVIDIA GeForce RTX 3090 GPU with 24 GB dedicated DRAM.

The testcases are from the EPFL combinational benchmark suite [3] and IWLS 2005 benchmarks [2], which include both arithmetic and control designs. To demonstrate the scalability and effectiveness of our parallelization strategy, we enlarge the designs by repeatedly applying the ABC command *double*, which is a common method used in prior studies on parallel logic synthesis and verification [15, 21, 22]. The statistics of them are presented in Table 1. Nangate 45nm library[1] is selected as the technology library.

All testcases are first mapped using the ASIC mapper in ABC with the command *&nf* to obtain the initial mappings. We then compare UniMap’s results against ABC’s gate sizing algorithm executed with the command sequence *upsize; dsize*. The maximum cut size  $k$  and the number of priority cuts per node  $C$  are respectively set as 6 and 8. The base weight  $W_{base}$  and area weight  $\beta$  are set as 0.2 and  $1 \times 10^{-4}$ . All results are evaluated with command *stime* in ABC and pass equivalence checking.

### 4.2 Computational Results

The comparison between UniMap and ABC is presented in Table 1. Note that “#Nodes” and “Levels” denote the number of nodes and topological levels of the input AIGs. Columns “Delay(ps)”, “Area( $\mu m^2$ )” and “#Gates” represent the delay,

**Table 2: Ablation study on sizing and remapping**

| Method  | Sizing | Remapping | Delay(ps) | Area( $\mu\text{m}^2$ ) |
|---------|--------|-----------|-----------|-------------------------|
| Geomean | -      | -         | 18348.45  | 6064381                 |
|         | ✓      | -         | 11150.25  | 6351833                 |
|         | -      | ✓         | 15505.52  | 5797763                 |
|         | ✓      | ✓         | 10380.55  | 6103356                 |

area and total number of gates of the mapped circuits, respectively. Column “Time(s)” displays the execution time which does not include the file I/O time.

**4.2.1 Comparison with ABC.** Table 1 presents the results mapped with Nangate 45nm library. On average, UniMap achieves 8% reduction in delay and 4% reduction in area. For nearly all the cases, UniMap provides superior timing performance, except for case vga\_lcd\_5xd. Notably, for voter\_9xd, UniMap achieves a 20% improvement in area and an 8% improvement in delay.

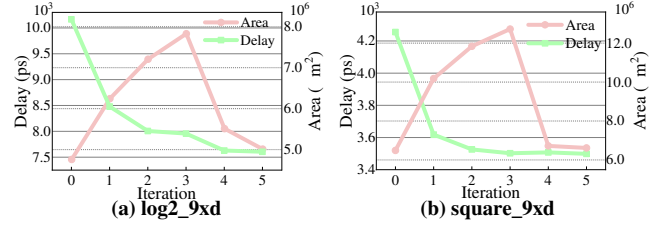
It is worth emphasizing that for some cases, such as sqrt\_7xd and voter\_9xd, UniMap produces results with smaller area than the initial mappings. This outcome is unattainable for pure gate sizing algorithms, since those methods can only upsize the gates for a solution already using the smallest available cell sizes. In contrast, UniMap integrates remapping and sizing, allowing it to explore other cell types and further reduce the area.

For cases like ac97\_ctrl\_10xd and vga\_lcd\_5xd, the improvements are relatively modest. This is primarily because these designs have shallow logic depth and limited mapping options per node, making remapping less effective.

In terms of runtime, UniMap completes incremental mapping for netlists exceeding five million gates within 15 seconds. Compared to ABC, UniMap achieves an average 60.3 $\times$  speedup, demonstrating its high efficiency. In particular, for sin\_10xd and multiplier\_9xd, UniMap reaches 249 $\times$  and 142 $\times$  speedups, respectively. Note that UniMap is in GPU parallelism, but it runs deterministically, which is enabled by our algorithmic design.

**4.2.2 Ablation Study.** To demonstrate the effectiveness of integrating sizing and remapping in our algorithm, we conduct an ablation study by selectively disabling the two components. Specifically, when sizing is disabled, remapping is restricted to use the smallest cell for each type. The results are shown in Table 2. Sizing alone achieves performance comparable to ABC’s sizing, but UniMap further reduces delay and area by 8% and 4%, respectively.

It is interesting that remapping alone can reduce delay and area of the initial mapping by 15% and 4%, respectively. Since only the smallest cells are used, this result highlights

**Figure 6: Procedure analysis.**

the effectiveness of our framework in improving ABC’s mapping. As expected, its delay is inferior to sizing alone and UniMap due to the use of minimum-sized cells with limited drive strength. Nevertheless, remapping provides substantial area improvement.

By combining these two strategies, UniMap achieves simultaneous improvements in both delay and area compared with ABC’s sizing.

**4.2.3 Procedure analysis.** To analyze the contribution of each optimization phase in our algorithm, we record the delay and area after each iteration during the mapping process. Figure 6 shows the results on log2\_9xd and square\_9xd. It can be observed that both delay optimization and area optimization phases contribute to the overall performance improvement. The delay optimization phase primarily focuses on reducing delay, and the area will be increased at this stage. While the area optimization phase effectively decreases area without significantly impacting delay. This demonstrates the effectiveness of our two-phase optimization strategy in achieving a balanced improvement in both delay and area.

## 5 Conclusion

In this paper, we propose UniMap, a GPU-accelerated incremental ASIC technology mapper that jointly considers sizing and remapping. Specifically, we introduce an evaluation framework equipped with two effective gain functions to assess the local regions of candidate configurations. To ensure the accuracy of timing evaluation, we propose incremental STA and AIG-based levelization. Furthermore, two deterministic timing update strategies are developed to guarantee consistent incremental STA under parallel execution. Finally, all components are integrated into a fine-grained GPU acceleration framework. Experimental results on public benchmarks demonstrate that UniMap achieves an average of 8% reduction in delay and 4% reduction in area compared to the state-of-the-art tool ABC, while providing a 60.3 $\times$  speedup through GPU acceleration.

## References

- [1] 2008. Nangate 45nm Open Cell Library. <http://www.nangate.com>

- [2] Christoph Albrecht. 2005. IWLS 2005 benchmarks. In *International Workshop for Logic Synthesis (IWLS)*, Vol. 9.
- [3] Luca Amarù, Pierre-Emmanuel Gaillardon, and Giovanni De Micheli. 2015. The EPFL combinational benchmark suite. In *Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS)*.
- [4] Robert Brayton and Alan Mishchenko. 2010. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification: 22nd International Conference, CAV 2010, Edinburgh, UK, July 15-19, 2010. Proceedings 22*. Springer, 24–40.
- [5] Alessandro Tempia Calvino and Giovanni De Micheli. 2023. Technology mapping using multi-output library cells. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [6] Alessandro Tempia Calvino, Giovanni De Micheli, Alan Mishchenko, and Robert Brayton. 2024. Enhancing Delay-Driven LUT Mapping with Boolean Decomposition. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [7] Alessandro Tempia Calvino, Alan Mishchenko, Herman Schmit, Ethan Mahintorabi, Giovanni De Micheli, and Xiaoqing Xu. 2023. Improving standard-cell design flow using factored form optimization. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [8] Satrajit Chatterjee, Alan Mishchenko, Robert K Brayton, Xinning Wang, and Timothy Kam. 2006. Reducing structural bias in technology mapping. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 25, 12 (2006), 2894–2903.
- [9] Chung-Ping Chen, Chris CN Chu, and DF Wong. 1998. Fast and exact simultaneous gate and wire sizing by Lagrangian relaxation. In *Proceedings of the 1998 IEEE/ACM international conference on Computer-aided design*. 617–624.
- [10] Chandrabhasan Reddy Chigarapally, Harshwardhan Nitin Bhakkad, Animesh Basak Chowdhury, Chandan Karfa, and Sukanta Bhattacharjee. 2024. LEAP: Learning guided Quality Cut selection for faster Technology Mapping. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–6.
- [11] Andrea Costamagna, Alessandro Tempia Calvino, Alan Mishchenko, and Giovanni De Micheli. 2025. Area-Oriented Optimization After Standard-Cell Mapping. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 1112–1119.
- [12] Yufan Du, Zizheng Guo, Yibo Lin, Runsheng Wang, and Ru Huang. 2024. Fusion of global placement and gate sizing with differentiable optimization. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [13] Guilherme Flach, Tiago Reimann, Gracieli Posser, Marcelo Johann, and Ricardo Reis. 2014. Effective method for simultaneous gate sizing and  $\alpha$  assignment using lagrangian relaxation. *IEEE transactions on computer-aided design of integrated circuits and systems* 33, 4 (2014), 546–557.
- [14] Zizheng Guo, Tsung-Wei Huang, and Yibo Lin. 2020. Gpu-accelerated static timing analysis. In *Proceedings of the 39th international conference on computer-aided design*. 1–9.
- [15] Shiju Lin, Jinwei Liu, Tianji Liu, Martin DF Wong, and Evangeline FY Young. 2022. NovelRewrite: Node-level parallel AIG rewriting. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 427–432.
- [16] Shiju Lin, J Liu, and MD Wong. 2024. 'InstantGR: Scalable GPU parallelization for global routing. In *2024 ACM/IEEE International Conference On Computer Aided Design (ICCAD)*.
- [17] Junfeng Liu, Liwei Ni, Lei Chen, Xing Li, Qinghua Zhao, Xingquan Li, and Shuai Ma. 2024. A Delay-Driven Iterative Technology Mapping Framework. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [18] Junfeng Liu, Liwei Ni, Xingquan Li, Min Zhou, Lei Chen, Xing Li, Qinghua Zhao, and Shuai Ma. 2023. Aimap: Learning to improve technology mapping for asics via delay prediction. In *2023 IEEE 41st International Conference on Computer Design (ICCD)*. IEEE, 344–347.
- [19] Lixin Liu, Bangqi Fu, Martin D. F. Wong, and Evangeline F. Y. Young. 2022. Xplace: An Extremely Fast and Extensible Global Placement Framework. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*.
- [20] Siting Liu, Yuan Pu, Peiyu Liao, Hongzhong Wu, Rui Zhang, Zhitang Chen, Wenlong Lv, Yibo Lin, and Bei Yu. 2022. Fastgr: Global routing on cpu-gpu with heterogeneous task graph scheduler. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 7 (2022), 2317–2330.
- [21] Tianji Liu, Lei Chen, Xing Li, Mingxuan Yuan, and Evangeline FY Young. 2024. FineMap: A fine-grained GPU-parallel LUT mapping engine. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 392–397.
- [22] Tianji Liu and Evangeline FY Young. 2023. Rethinking AIG resynthesis in parallel. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [23] Yi-Chen Lu, Kishor Kunal, Geraldo Pradipta, Rongjian Liang, Ravikishore Gandikota, and Haoxing Ren. 2025. LEGO-Size: LLM-Enhanced GPU-Optimized Signoff-Accurate Differentiable VLSI Gate Sizing in Advanced Nodes. In *Proceedings of the 2025 International Symposium on Physical Design*. 152–162.
- [24] Qiang Ma and Evangeline FY Young. 2010. Multivoltage floorplan design. *IEEE transactions on computer-aided design of integrated circuits and systems* 29, 4 (2010), 607–617.
- [25] Wai-Kei Mak and Evangeline FY Young. 2002. Temporal logic replication for dynamically reconfigurable FPGA partitioning. In *Proceedings of the 2002 international symposium on Physical design*. 190–195.
- [26] Alan Mishchenko, Sungmin Cho, Satrajit Chatterjee, and Robert Brayton. 2007. Combinational and sequential mapping with priority cuts. In *2007 IEEE/ACM International Conference on Computer-Aided Design*. IEEE, 354–361.
- [27] R Murgai. 1999. On the complexity of minimum-delay gate resizing/technology mapping under load-dependent delay model. In *Workshop Handouts, International Workshop on Logic Synthesis*. 209–211.
- [28] Walter Lau Neto, Matheus T Moreira, Yingjie Li, Luca Amarù, Cunxi Yu, and Pierre-Emmanuel Gaillardon. 2021. SLAP: A supervised learning approach for priority cuts technology mapping. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 859–864.
- [29] Ankur Sharma, David Chinnery, Sarvesh Bhardwaj, and Chris Chu. 2015. Fast Lagrangian relaxation based gate sizing using multi-threading. In *2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 426–433.
- [30] Yang Sun, Tianji Liu, Martin DF Wong, and Evangeline FY Young. 2024. Massively Parallel AIG Resubstitution. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [31] Linfu Xiao, Subarna Sinha, Jingyu Xu, and Evangeline FY Young. 2010. Fixed-outline thermal-aware 3D floorplanning. In *2010 15th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 561–567.
- [32] Zhenxuan Xie, Lixin Liu, Tianji Liu, and Evangeline FY Young. 2025. ExactMap: Enhancing Delay Optimization in Parallel ASIC Technology Mapping. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.